

EXPERIMENT 9: BREADTH-FIRST SEARCH (BFS)

AIM

To develop a Python program that implements the Breadth-First Search (BFS) algorithm for traversing a graph and visiting all reachable nodes in level-by-level order.

OBJECTIVE

- To understand the concept of the **Breadth-First Search (BFS)** algorithm.
- To traverse a graph using a queue.
- To visit each node exactly once.
- To display the BFS traversal order.
- To find the shortest path in an unweighted graph.

ALGORITHM

Step 1: Start.

Step 2: Create the graph using an adjacency list.

Step 3: Create an empty queue and an empty set for visited nodes.

Step 4: Insert the starting node into the queue and mark it as visited.

Step 5: Remove the front node from the queue.

Step 6: Display the current node.

Step 7: Visit all adjacent unvisited nodes.

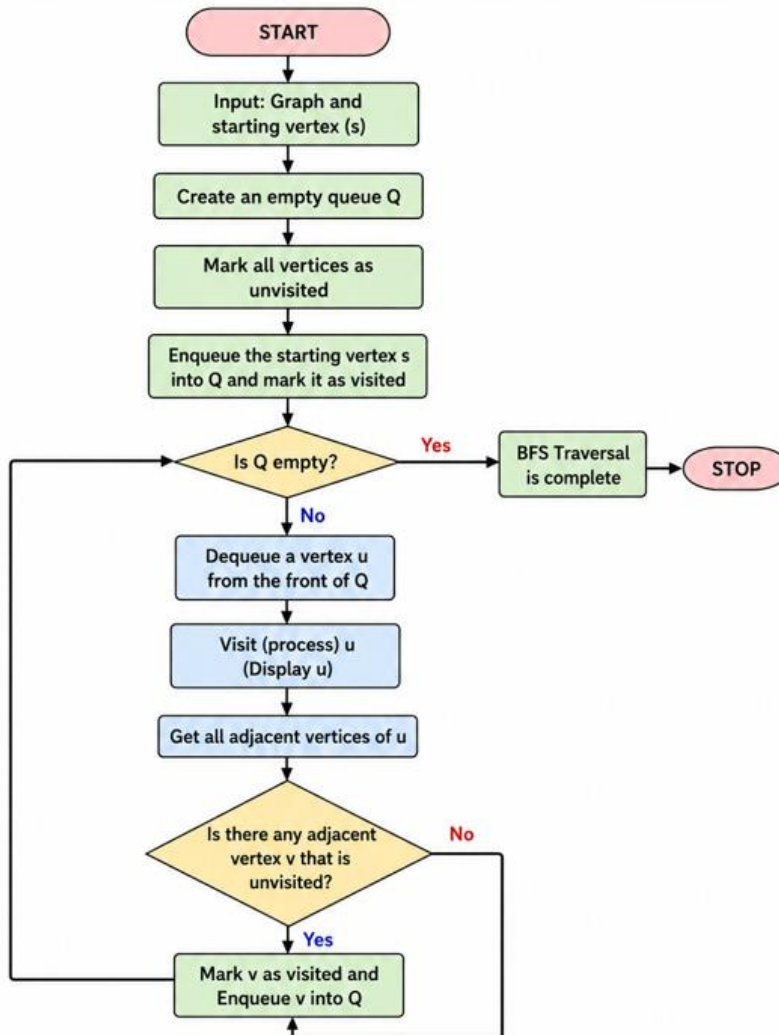
Step 8: Mark each adjacent node as visited and insert it into the queue.

Step 9: Repeat Steps 5–8 until the queue becomes empty.

Step 10: Stop.

FLOW CHART

BREADTH FIRST SEARCH (BFS) ALGORITHM – FLOWCHART



SOURCE CODE

```
from collections import deque
```

```
graph = {  
    'A': ['B', 'C'],  
    'B': ['D', 'E'],  
    'C': ['F'],  
    'D': [],
```

```
'E': ['F'],  
'F': []  
}
```

```
visited = set()  
queue = deque()
```

```
def bfs(start):  
    visited.add(start)  
    queue.append(start)  
  
    while queue:  
        node = queue.popleft()  
        print(node, end=" ")  
  
        for neighbor in graph[node]:  
            if neighbor not in visited:  
                visited.add(neighbor)  
                queue.append(neighbor)  
  
print("BFS Traversal:")  
bfs('A')
```

OUTPUT

```
BFS Traversal:  
A B C D E F
```

RESULT

The Python program successfully performs Breadth-First Search (BFS) on the given graph and displays the traversal order by visiting each node exactly once in level-by-level order using a queue.

